# Project status and handoff

Snapshot of 2026-11-02. Read this before changing anything. How to *use* the engine and tools is in README.md; this file covers what exists, what is committed, what is tested, and how work is done here.

## 1. Ground rules

- **Never commit unless the owner asks.** Never add AI/Claude attribution (`Co-Authored-By`, "Generated with" lines) to commits or PRs.
- **Every strength change is proven by SPRT** (`tools/sprt.py`, see README): one change, one test, one commit. H0 (not stronger) means revert, however good the idea seemed. A change whose bench node count is unchanged doesn't alter play and needs no SPRT (except time-management changes, which the fixed-depth bench can't see: use `--force`).
- **Two evaluation lines, kept apart** (owner's decision):
  - `Eval=PeSTO`: borrowed tables, the fixed reference opponent. Stays pure; it never gets tuned terms and never supplies training data.
  - `Eval=Learned` → `Eval=NNUE`: self-taught. Starts from material only and trains **only on its own self-play games**. Data files carry a `.meta` source line; the tuner and NNUE trainer refuse anything not played by the Learned line. Don't bypass this.
- **No opening book and no tablebases in play.** Every move is the engine's own. Opening books are only for SPRT testing.
- **Explanation and style code stay outside the core search.** Teaching features are built on top over UCI (`tools/coach/`). Styles (Contempt, Creative, Practical, Variety) are a layer at the root, after the search.
- **Never commit the Lichess token.** Upgrading an account to a bot account is irreversible; only the owner does it.
- Before any test, run `make test` (sanitized debug build) and `make perft` for move generation changes. Don't run heavy CPU work while an SPRT is running: OS stalls cause time forfeits at fast time controls.

## 2. Repository map

| Path | What |
|---|---|
| `src/types.h`, `bitboard.*`, `position.*`, `movegen.*`, `perft.cpp` | Board (bitboards, magics), make/unmake, legal move generation, SEE, SAN parsing |
| `src/search.*` | Iterative deepening, PVS alpha-beta, quiescence, move ordering, time management, Lazy SMP, root style hook |
| `src/tt.*` | Lockless transposition table (XOR-checked two-word slots) |
| `src/eval.*` | PeSTO, Learned (PSQT + 25 classical term groups, linear, tunable), eval breakdown, `threats` |
| `src/nnue.*` | NNUE inference (768 → 256×2 → 1, SCReLU), incremental accumulator updated by Position |
| `src/datagen.*` | Self-play training data (32-byte packed positions + `.meta`) |

| Path | What |
|---|---|
| `src/tune.*` | Texel tuner (full-batch Adam, fitted K, weight normalization) |
| `src/style.*` | Contempt/Creative/Practical/Variety root layer, `material_dip` |
| `src/motifs.*` | Tactical motif detection for teaching (fork, pin, skewer, discovered attack/check, mate, hanging piece, promotion, sacrifice) |
| `src/uci.cpp` | UCI loop, options, extra commands (`bench`, `eval`, `threats`, `datagen`, `tune`, `motifs`, `sanline`, `fen` ...) |
| `src/tests.cpp` | Self-tests (`make test`) |
| `tools/sprt.py` | SPRT/fixed-game matches via fastchess (`~/dev/tools/bin/fastchess`) |
| `tools/learn.py` | Runs Learned-line generations: self-play → tune/train → optional match |
| `tools/nnue/train.cpp` | NNUE trainer (`make trainer` → `build/release/nnue-train`) |
| `tools/coach/coach.py` | Coach: analyze, puzzles, selfplay, openings, play (stdlib only) |
| `tools/lichess_bot.py` | Lichess bot (stdlib only, needs `LICHESS_TOKEN`) |
| `tools/play/` | Browser play page: `server.py` (stdlib) + `index.html` + vendored chess.js 0.10.3 (BSD-2). Multiple zero-time premoves, auto-save to `games/`, Analyse button (coach) |
| `tools/uci_selfplay.py` | Older self-play helper |
| `evals/learned/genN.txt`, `evals/nnue/netN.nnue` | Trained weights (small, meant to be committed) |
| `books/8moves_v3.pgn` | SPRT opening book (gitignored) |
| `build/` | **Gitignored.** Builds, SPRT runs, training data, and the uncommitted patch stack |

## 3. History

| Commit | Content |
|---|---|
| `79d1bc8` | Phases 0–3: board, movegen, perft, UCI, alpha-beta, quiescence, PeSTO |
| `13dcb16` | Phase 4: SPRT testing with fastchess |
| `d9900da` | Phase 5 steps 01–14, committed by the owner as one commit (its title only mentions SEE/TT): 01 time reserve, 02 TT, 03 PVS, 04 null move, 05 killers/history, 06 LMR, 07 SEE, 08 RFP, 09 check extension, 10 aspiration windows, 11 LMP/futility, 12 qsearch TT, 13 IIR, 14 bench depth. Only step 01 was SPRT'd (undecided, no measurable cost; fixed the fast-TC time forfeits). |

Everything after `d9900da` is **uncommitted**, in the working tree.

## 4. Uncommitted work: the patch stack

Each change was saved as its own step without committing, so it can be tested and committed one by one. The tooling:

- `build/steps/snap.sh <phase-dir> <NN-name>` records the working tree as a git tree object (temp index, no commit), appends `<tree-id> <name>` to `build/steps/trees`, writes `build/<phase-dir>/<NN-name>.diff` against the previous step, and copies the release build to `build/sprt/engines/p<N>-<NN-name>` for SPRT.
- The first line of `build/steps/trees` (phase5/14) equals the tree of `d9900da`.
- The working tree = the last step (teach/03) plus trained weight files (`evals/learned/gen1-3.txt`, `evals/nnue/net1.nnue`).

| # | Step | What it does | Changes play? |
|---|---|---|---|
| 1 | phase5/15-mate-distance | Mate distance pruning | No (bench identical) |
| 2 | phase5/16-improving | `improving` flag (static eval vs 2 plies ago) in RFP, LMP, LMR | Yes |
| 3 | phase5/17-countermove-conthist | Countermove table + continuation history (1- and 2-ply) in ordering | Yes |
| 4 | phase5/18-history-lmr | LMR reduction adjusted by history (`hist/8192`) | Yes |
| 5 | phase5/19-singular-extension | Singular extensions (depth ≥ 8) with multicut | Yes |
| 6 | phase5/20-time-stability | Soft time scaled by best-move stability and score drops | Yes (time only) |
| 7 | phase5/21-qsearch-delta | Delta pruning in quiescence | Yes |
| 8 | phase6/01-eval-terms | 25 classical term groups for **Learned only** (PeSTO bench unchanged) | No (PeSTO) |
| 9 | phase6/02-datagen-texel | `datagen` self-play data, `tune` Texel tuner, `datadump` | No |
| 10 | phase8/01-nnue | NNUE inference, `Eval=NNUE`, trainer `tools/nnue/train.cpp` | No (PeSTO) |
| 11 | phase9/01-lazy-smp | `Threads` option, lockless TT | No at 1 thread |
| 12 | phase9/02-lichess-bot | `tools/lichess_bot.py` | No |
| 13 | style/01-contempt-style | `Contempt`, `Style=Creative/Practical` root layer | No at defaults |
| 14 | teach/01-san-variety | SAN input, `sanline`, `fen`, `Variety` option | No at defaults |
| 15 | teach/02-coach | `tools/coach/coach.py` | No |
| 16 | phase6/03-normalize-weights | `Eval::normalize`: folds PSQT/mobility averages into piece values (same evals) | No |
| 17 | teach/03-motifs | `motifs` command, used by the coach | No |

Current bench (`make bench`, default depth): **2855916 nodes**. All of `make test`, `make perft` and a ThreadSanitizer run of Lazy SMP passed on the final step.

**Fragility:** the trees are unreachable git objects and `build/` is gitignored, so the stack lives only on this disk and `git gc` can eventually prune the trees (the `.diff` files remain). Committing, or at least a WIP branch backup, removes this risk.

## 5. Testing status

### SPRT of the search steps (running now)

`build/steps/queue.sh` runs each step against the previous one: 8+0.08, `8moves_v3`, bounds [0, 5], concurrency 6. Summary log: `build/steps/queue.log`; full output: `build/steps/out/<name>.out`; games: `build/sprt/runs/`; all results also in `build/sprt/history.txt`. The script ends with a macOS notification sound.

| Step | Result |
|---|---|
| 15 mate distance | Skipped: identical bench (no effect at test depths; keep it, it's correct) |
| 16 improving | **H1**: +17.95 ±8.71 Elo, 3196 games |
| 17 countermove/conthist | **H1**: +10.83 ±6.50 Elo, 5712 games |
| 18 history LMR | **H1**: +54.37 ±15.88 Elo, 1050 games |
| 19 singular extension | Running (≈ +12 Elo at 3160 games, LLR 1.77/2.94) |
| 20 time stability | Queued (`--force`, time-only change) |
| 21 qsearch delta | Queued |

Note: the Mac hibernated on battery during step 19 (14:39–17:25); 6 games in flight were lost on time (4 by base, 2 by dev). Negligible; the test continued.

**Plan if a step gets H0:** drop that step, rebuild the later steps without it (re-apply their diffs onto the previous accepted step, re-snap), and re-test those later steps.

### Learning matches (200 games each, same binary, different weights)

| Match | Result |
|---|---|
| Learned gen1 vs gen0 (material only) | +391.6 ±80.5 (175 W, 13 L, 12 D) |
| Learned gen3 vs gen1 | +372.3 ±74.7 (172 W, 14 L, 14 D) |
| NNUE net1 vs Learned gen3 | −181.7 ±49.8 (39 W, 135 L, 26 D) |
| NNUE net1 vs PeSTO | −164.1 ±51.7 (40 W, 128 L, 32 D) |

The Learned line learns well. net1 is weak, as expected of a first net trained on few positions (validation loss 0.0742 → 0.0127): it needs more generations (`learn.py --nnue --generations N`), ideally after the SPRT queue finishes so they don't compete for CPU. gen3's middlegame piece values are inflated (knight 718, queen 2846 mg) while endgame values recover; watch this over generations.

### Not tested in games

- Steps 8–17 don't change PeSTO play at defaults, so they need no SPRT; they are covered by unit tests.
- Style=Creative/Practical and Contempt: unit-tested only. They cost Elo against engines by design and should be judged against weaker or noisy opponents, or humans.
- Lazy SMP strength at >1 thread: TSan-clean, not measured (e.g. 2 vs 1 thread SPRT).
- Lichess bot: never run live (needs the owner's bot account and token).

- Absolute rating: unknown. Only self-play so far. A gauntlet against engines with known CCRL ratings, or the Lichess bot, would give one.

## 6. Open items

Needs the owner's decision or action:

1. **Commits**: one commit per step (planned; see below) or grouped. Not authorized yet.
2. **Phase 7, "Freeze v1"**: tag a v1 once the SPRT queue is done.
3. **PeSTO+**: should PeSTO also get tuned classical terms? Currently no (it stays the fixed reference).
4. **Lichess**: create a bot account, token with `bot:play`, upgrade (irreversible).

Everything else planned is in section 9.

## 7. How to commit the stack (when asked)

The trees are full snapshots, so commits can be built without touching the working tree:

```
parent=$(git rev-parse HEAD)
tail -n +2 build/steps/trees | while read tree name; do
    parent=$(git commit-tree $tree -p $parent -m "<message for $name>")
    echo "$parent $name"
done
git reset --soft $parent     # HEAD = last step; the working tree stays as it is
# then commit evals/ (trained weights) separately
```

One commit per step, message with what changed, the bench, and the SPRT summary for play-affecting steps. **No AI attribution.** If a step was dropped after H0, the stack must be rebuilt first (section 5).

## 8. Environment

- macOS on Apple M1 (8 cores: 4 performance, 4 efficiency), 8 GB RAM, ~22 GB free disk; keep training data compact (`build/data/learned/`, 163 MB now).
- `clang++` C++20; fastchess at `~/dev/tools/bin/fastchess`; En Croissant installed (engine added as "My Engine" → `build/release/engine`). En Croissant allows only one premove and forgets unsaved games on close; `tools/play/server.py` has neither limitation.
- Long runs: keep the Mac on the charger. `caffeinate` only prevents sleep on AC power, and closing the lid sleeps the machine regardless.
- The SPRT machine plays about 850 games an hour at 8+0.08 with concurrency 6.

## 9. Future plan

Ordered by dependency. Each item lists its goal, the steps, where the code goes, how it is accepted, and a rough size. Items marked **(owner)** need a decision or action from the owner first. Unless noted, every item that changes play is one SPRT-tested commit per step (section 1).

### Phase A: close out the current work (next)

**A1. Finish the SPRT queue.** Wait for steps 19–21 (`build/steps/queue.log`).

- If all pass: the stack is ready to commit.
- If a step gets H0: remove it, re-apply the later diffs onto the previous accepted step, re-snap with `snap.sh`, and re-test the later steps against the new predecessors.
- Accept: every play-affecting step 16–21 has H1, or was removed.

**A2. Commit the stack (owner).** One commit per step via the recipe in section 7, then a commit for `evals/` (trained weights), `docs/STATUS.md` and `CLAUDE.md`. Push.

- Accept: `git status` clean apart from ignored files; each commit's message has the bench, plus the SPRT summary for play-affecting steps.

**A3. Phase 7, "Freeze v1" (owner).** Tag the commit as `v1.0` and keep a copy of the binary as the long-term baseline (`build/sprt/engines/v1.0`). Later work is SPRT'd against the previous commit as usual, and periodically against v1.0 to track total progress.

- Size: minutes.

### Phase B: measure real strength

**B1. Rating gauntlet.** Play v1 against 4–6 open-source engines with published CCRL ratings bracketing the expected level (about 2000–2600 CCRL Blitz), 200+ games each at 8+0.08 with the SPRT book, all via fastchess. Compute a rating with fastchess's output or Ordo (anchored on the opponents' CCRL ratings).

- Code: a small `tools/gauntlet.py` (opponent list, fastchess call, results table), or plain fastchess commands documented in the README.
- Accept: a rating with an error bar under ±40, recorded in `docs/STATUS.md`.
- Size: ½ day of setup, overnight to run.

**B2. Lazy SMP scaling.** Fixed-game matches with `Threads=2` vs `Threads=1` and `Threads=4` vs `2` (fewer concurrent games so threads don't oversubscribe the 4 performance cores).

- Accept: 2 threads ≥ +40 Elo over 1. If not, profile TT contention and helper depth skipping (`src/search.cpp` helpers).
- Size: overnight per pair.

### Phase C: grow the Learned line

**C1. More hand-crafted generations.** `python3 tools/learn.py --generations N --match 200` until a generation gains under ~10 Elo over its predecessor. Watch the inflated middlegame piece values in gen3 (`evalparams`); if they keep growing, try a lower `--lr` or larger `--lambda` mix of game results.

- Accept: the latest generation is the best of the line in a 200-game round-robin.
- Size: about 30–60 min per generation.

**C2. NNUE generations.** Train each net on more positions and on its own games:

1. Generate data with the latest net playing itself (`learn.py --nnue`), keeping the `.meta` source checks.
2. Grow the training set to 30–100 M positions (32 bytes each: 1–3 GB; fits the free disk).
3. Start each net from the previous one (`--init`).

- Accept: netN beats the latest hand-crafted generation, then beats PeSTO (SPRT, not just 200 games).
- Size: hours per generation.

**C3. Faster NNUE inference.** Vectorize the accumulator updates and the output layer with ARM NEON (Apple M1), with the scalar code kept as fallback. Then try a larger hidden layer (512) and output buckets by piece count.

- Code: `src/nnue.cpp`, `tools/nnue/train.cpp` (format change → bump the `ENNUE1` magic).
- Accept: unit test incremental == from-scratch still passes; same bench; NPS up; SPRT H1 for the speedup at `Eval=NNUE`.

**C4. Make NNUE the default (owner).** Once a net clearly beats PeSTO, decide whether the default `Eval` becomes NNUE. PeSTO stays available as the reference.

**C5. PeSTO+ (owner, open question).** Whether PeSTO also gets tuned classical terms. Only if the owner wants it; it would have to be tuned on separate data, never mixed into the Learned line.

### Phase D: more search strength

Classical ideas still missing. Each is one SPRT step against the previous commit, in roughly this order of expected gain:

1. Capture history for ordering captures (and in qsearch).
2. SEE pruning of bad captures and quiets in the main search at low depth.
3. Correction history: adjust the static eval by the stored error of past searches from the same pawn structure.
4. ProbCut.
5. History gravity (bonus/malus scaled so values saturate) and a history malus for quiets that didn't cut.
6. Razoring at depth 1–2.
7. Parameter tuning with SPSA (margins, LMR table constants, history divisors): a `tools/spsa.py` that runs fastchess with perturbed UCI options. This needs the parameters exposed as hidden UCI options first.

- Accept: H1 each; simplifications with `--bounds simplify`.

### Phase E: playing styles, tested properly

**E1. Test against fallible opponents.** Creative/Practical are meant to beat humans, not engines. Measure them against weaker or noisy versions of the engine (depth-limited, or a version that sometimes picks a random move among its top 3).

- Code: a "noisy" UCI option (e.g. `Blunder` %) for the opponent, used only in tests.
- Accept: each style scores better than Objective against the noisy opponent, without losing more than ~20 Elo against the full engine.

**E2. WDL model for Practical.** Replace the fixed 80 cp softmax with a win/draw/loss model fitted on self-play results (score → win probability by game phase).

- Code: fitting in Python from `datadump` output; constants in `src/style.cpp`.

**E3. UCI_Elo / skill levels.** Let people play a weaker version: limit nodes and depth, and add calibrated randomness, mapped to an approximate Elo by the B1 gauntlet.

### Phase F: teaching

**F1. Save games from** `coach.py play` to a PGN (about 10 lines). The browser play page (`tools/play/`, added 2026-11-02) already saves every game and analyses it; the terminal mode still doesn't.

**F2. Positional explanations.** Use the Learned eval's term breakdown (`eval`) to say *why* a move is good or bad positionally: compare terms before and after a move ("weakens your king's pawn shield", "creates a passed pawn", "your rook gets the open file"). Coach templates in `tools/coach/coach.py`; no search changes.

- Accept: on a sample of analysed games, the stated term changes match the eval breakdown.

**F3. LLM rephrasing layer (optional, needs an API key).** Send the coach's facts (classification, motifs, best line, term changes) to an LLM that may only rephrase them in friendly language, never add analysis. A Claude model from the current family, key from an environment variable, off by default, with the plain template text as fallback.

- Code: `tools/coach/llm.py`, a `--explain llm` flag.

**F4. Opening explorer and repertoire.** Grow `coach.py openings` into a browsable HTML tree of what the engine plays (from Variety self-play), and of the owner's own games.

**F5. Puzzle training mode.** Serve puzzles from the owner's own mistakes (`puzzles`) one at a time in the terminal, with retries and a success record.

### Phase G: Lichess bot go-live (owner)

1. The owner creates a new account, a `bot:play` token, and upgrades it (irreversible).
2. Run `tools/lichess_bot.py` with `LICHESS_TOKEN` set, starting with casual games and short time controls, `Threads=2`.
3. Watch the first ~50 games for crashes, flags and disconnects; fix the bot as needed.
4. Later: let the bot challenge other bots, use a chosen style, and keep logs.

- Accept: 100 games without a crash or a time forfeit; a Lichess bot rating (another strength measure for B1).

### Phase H: maintenance

- **CI**: a GitHub Actions workflow running `make test` and `make perft` on every push (Linux and macOS).
- **Back up the uncommitted state** whenever it lasts more than a day (WIP branch), since `build/` is gitignored.
- **Keep docs current**: update this file and the README after every phase.